

Lecture 11 — From a Messy File to a Gradebook

Data cleaning, grading schemes, and the start of an object-oriented design

Amir Farbin

The problem

Today's problem

- A TA hands you a dump of everyone's scores. Grades are due.
- No software, no time, and the file is a mess
- A silly problem **on purpose**: everything real is in how we solve it
- Three levels today:
 - get the data usable
 - decide what a grade *means*
 - build something that survives being used again

→ open `Data-1401-Grades.csv`

Rule zero: look at the file

- `!ls -lh` — how big? Can I even open it?
- `!cat` — what is actually in there?
- What's wrong here:
 - first line is grading notes, not a header
 - words where numbers belong: *no solution, did not submit*
 - scores written 68-sent — the TA's note that it was emailed
 - blank cells everywhere

→ cells 2, 4

Cleaning

Fix it from the shell, not by hand

- Never edit the data file by hand — you can't repeat it, and you can't explain it
- `!wc -l` to count, `!tail` to drop the bad first line, `>` to write a new file
- Keep the original. Write `Data-1401-Grades-Fixed.csv`
- Same tools they just used in the Lab 5 command-line session

→ cells 9, 11, 13, 15

pandas reads it properly

- `pd.read_csv` — now the header is a header
- **DataFrame** = a table; **Series** = one column
- A Series looks like a NumPy array but carries labels
- `df["Lab 1"]` — one column; the column names came from the file

→ cells 17, 18, 20

Selecting rows

- `df[...]` is a very smart `__getitem__` — exactly Lecture 10's material
 - a string gives a column
 - a column of True/False gives rows
- Build a mask, apply it, get a new DataFrame back
- Assigning to a name that doesn't exist **creates a column**

→ cells 24, 25, 33

60–80% of the work is this

- Cleaning is most of the job, on every real dataset
- Fix the *format*, not the numbers
- Better: go back to whoever produced the file and fix it at the source
- Formats drift — look at the file every time, even when it's “the same” file

What the numbers say

Histogram first

- A column of scores is a **random variable**; a histogram estimates its distribution
- Always look before you compute a statistic
- This one isn't normal — it's lumpy
 - mixed populations: students who did the work and students who didn't
 - the CLT doesn't save you when it's a mixture
- Mean and σ describe a distribution **only when it's normal**

→ cells 37, 38, 47

What is a grade for?

- A grade groups students who performed similarly — that's all it is
- Straight scale: 90/80/70 — assumes mean 80, σ 10. Says who told you that?
- Curve: map the class onto a normal distribution
- Normalized percentile: rank, then cut
- **The same score must get the same letter.** Pick one scheme and defend it

Counting in ranges

- `in_range(a,b)` returns a *function* — a closure, as in Lab 3
- Every way of counting scores in a range is **histogramming**
- Grade boundaries are just **bin edges**
- `plt.hist(..., bins=[0,50,64,80,90,101])` — the scheme *is* the bin list

→ cells 51, 56, 72

Summarize and curve

- `grade_summary` — how many in each letter, as a fraction
- `min_grade` — what floor do you set, and why?
- `compute_curve` — shift and scale to a target mean and spread
 - the same shift-and-scale as Lab 3's distribution exercise
- A is capped at the max; no one gets more than the best score

→ cells 74, 75, 76, 80, 81

When it breaks: the `TypeError`

- Run the curve on Lab 1 and it **fails** — show the traceback
- Read it: comparing `str` and `float`
- `list(map(type, ...))` — the column is full of strings
- Why: comment rows made pandas read the whole column as text
- `fillna(0), astype` — now the numbers are numbers

→ cells 87, 89, 84, 85

Assigning letters

- `assign_grade` walks the boundaries from the bottom up
- `map` applies it down the column — or a plain loop; both fine
- Weighted total across labs and exams → the class grade

→ cells 93, 96, 98, 99, 102

Building a system

Dissecting the task

- Every step is a **family of functions with the same inputs and outputs**:
 - statistics: grades $\rightarrow$ mean, σ , min
 - boundaries: stats $\rightarrow$ cut points
 - assignment: boundaries + grades $\rightarrow$ letters
 - validation: letters $\rightarrow$ fractions per grade
- Same signature, many implementations $\rightarrow$ that's a **class**

From a notebook to production

- Day 1: cut and paste in a notebook; the client wants an answer tomorrow
- Then: do it a hundred times → write functions
- Then: new data, not just this file → generalize
- Then: hand it to a TA who doesn't have your context → it needs an interface
- Finally: production, for users who aren't you
- Effort buys **reliability**, not features

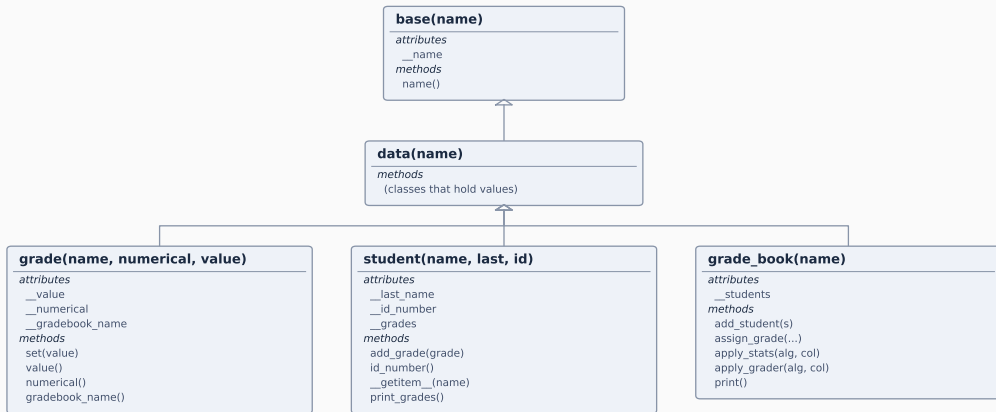
Why not just write it fast?

- Legacy code is the normal state: my experiment's ~7M lines of C++, 25 years old
- Interfaces outlive implementations — that's what you protect
- Data and algorithms stay in **separate classes**
 - the PDF format vs. the PDF reader
- Neither choice is obviously correct — avoid inventing generality you don't need

- Boxes: class name, attributes, methods
- Arrows: inheritance
- Draw it first — cheaper to erase a box than to refactor code
- Tools generate the skeleton from the diagram
- Lab 6 asks for exactly this, before any blackjack code

The design, before the code

Data classes: they hold values



Every class carries a name, so an instance can be found and reported on.

The design

Base classes

- `base(name)` — everything has a name, so bookkeeping and lookup work
- `data` — things that hold values
- `alg` — things that do work
- Empty classes are fine as scaffolding: they mark where behavior will go (persistence, verbosity) without committing yet
- `__name` is private; the mangled `_base__name` is Lecture 10's name mangling

→ cell 108

One grade class, or two?

One class, a flag inside

grade(name, numerical=True, value=None)

attributes

`__value`
`__numerical`
`__gradebook_name`
`__letter_grades`

methods

`set(value)`
`numerical()`
`__str__()`

Two subclasses

grade(name, value=None)

attributes

`__value`
`__gradebook_name`

methods

`set(value)`
`__str__()`

letter_grade

attributes

`__letter_grades`

methods

`set(value)`

numerical_grade

methods

`set(value)`

Neither is obviously correct. The flag means one class to hold, and a branch in every method. The subclasses remove the branches, and add more classes than the problem needs yet.

Representing a grade and a student

- grade: value, numeric or letter, which gradebook column it came from
- One class with a flag, or two subclasses? **Neither is obviously right**
- student: identity fixed at construction; grades added later
- add_grade refuses to overwrite — a bad value should fail loudly, near where it got in

→ cells 117, 119

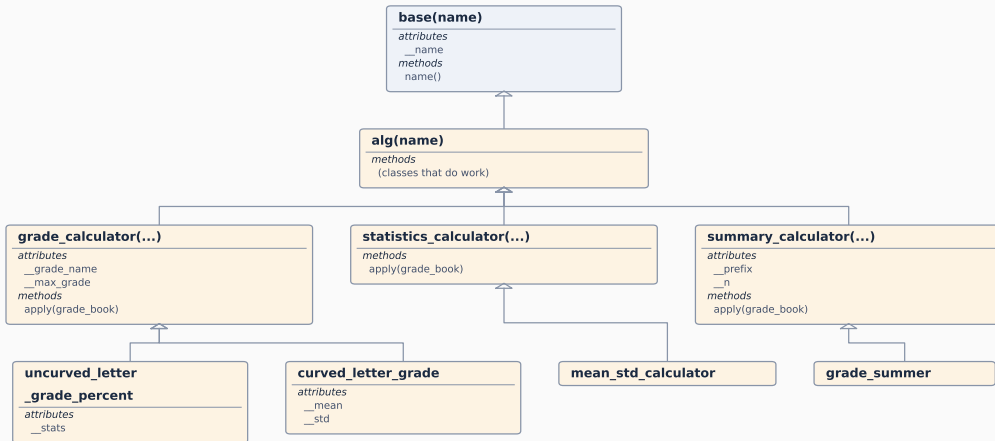
Algorithms as classes

- `grade_calculator` — configuration in the constructor, work in `apply`
- `uncurved_letter_grade_percent`, `curved_letter_grade` — same interface, different policy
- `mean_std_calculator` — statistics
- `summary_calculator` — sums the weighted total
- Each holds only its *settings*; the data lives elsewhere

→ cells 123, 124, 127, 130, 135

The calculators

Algorithm classes: same interface, different policy



Each subclass overrides apply(). Its own data is configuration, not the grades.

The gradebook

- `grade_book` holds students and their grades, and the results algorithms produce
- Load it from the DataFrame: rows $\rightarrow$ students, columns $\rightarrow$ grades
- `apply_stats`, `apply_grader`, `apply_summary` — run an algorithm over everyone
- Results get stored back in the book, so the next algorithm can use them

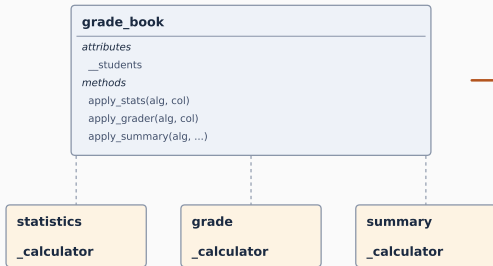
$\rightarrow$ cells 137, 140, 141, 142, 143

Where this is going

- It works. And the design already has a problem:
 - three kinds of calculator, three different `apply` methods
 - they differ only because we chose to make them differ
 - can't compose: curve, then normalize? Not without new code
- Lecture 12: collapse them into one `apply_calculator` interface
- That is the production-software lesson this chapter has been building to

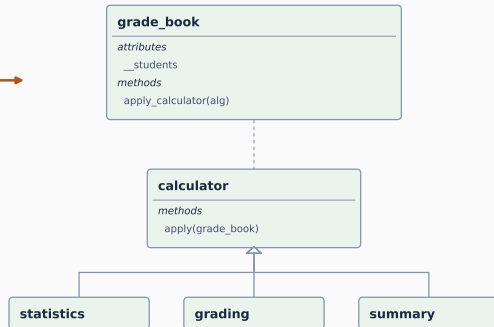
Lecture 12, in one picture

Lecture 11: three kinds, three methods



the caller has to know which kind it holds

Lecture 12: one interface



the grade book no longer cares